AIA8: Multimodal AI

Module Introduction and Theoretical Foundations

Multimodal AI integrates **text + image + audio + video** into unified representations, unlocking human-like perception. This module covers vision-language models (CLIP, Flamingo), unified architectures (ImageBind, Chameleon), and production deployment patterns for multimodal systems.

Theoretical Foundations:

1. **Unified Embedding Spaces**: Contrastive alignment across modalities
2. **Cross-Attention Fusion**: Late/early/mid fusion tradeoffs
3. **Scaling Laws**: Multimodal > unimodal at same compute
4. **Emergent Multimodal Reasoning**: Diagram understanding, video QA

Architecture Evolution:

text

CLIP (2021) → BLIP → Flamingo → Kosmos → GPT-4V → Gemini 1.5

(Figure 1: Unified multimodal embedding space)

Section 1: Contrastive Learning and CLIP Architecture

Contrastive Pretraining (Radford 2021):

text

Image-Text pairs (400M from web)

Loss: InfoNCE across batch

CLIP Loss Formulation:

text

L = L_image2text + L_text2image

L_image2text = -1/N Σ log[exp(sim(I_i,T_i)/τ) / Σ exp(sim(I_i,T_j)/τ)]

Vision Transformer (ViT) Encoder:

text

Patch embedding: Image → 196 patches (14×14)

Positional encoding + Transformer blocks

Output: [CLS] token or pooled

Complete CLIP Implementation:

python

```
class CLIPModel(nn.Module):
    def __init__(self, vision_backbone="ViT-B/32", text_dim=512):
        super().__init__()
        self.vision_encoder = VisionTransformer.from_pretrained(vision_backbone)
        self.text_encoder = TextTransformer(text_dim)
        self.logit_scale = nn.Parameter(torch.ones([]) * np.log(1/0.07))

    def forward(self, images, texts):
        image_features = self.encode_image(images)
        text_features = self.encode_text(texts)

        # Normalize
        image_features = F.normalize(image_features, dim=-1)
        text_features = F.normalize(text_features, dim=-1)

        # Similarity
        logits_per_image = self.logit_scale.exp() * image_features @ text_features.t()
        logits_per_text = logits_per_image.t()

        return logits_per_image, logits_per_text

    def encode_image(self, images):
        return self.vision_encoder(images)

    def encode_text(self, texts):
        return self.text_encoder(texts)
```

Zero-Shot Classification:

```
text
scores = image_features @ text_features["cat", "dog", "car"].T
pred = argmax(scores)
```

Section 2: Vision-Language Fusion Architectures

Fusion Strategies:

1. **Early Fusion**: Concatenate embeddings
2. **Late Fusion**: Independent encoders + cross-attention
3. **Mid Fusion**: Per-layer cross-attention

Flamingo Architecture (Alayrac 2023):

text

Perceiver Resampler: Compress 100k+ visual tokens → 64

Cross-attention: Text queries visual KV

LLaVA Architecture:

text

Vicuna LLM + CLIP Vision

Projection layer: Vision tokens → LLM space

LLaVA Forward Pass:

python

```python
class LLaVA(nn.Module):
    def __init__(self, vision_encoder, llm, vision_dim=1024, llm_dim=4096):
        super().__init__()
        self.vision_encoder = vision_encoder
        self.llm = llm
        self.mm_proj = nn.Linear(vision_dim, llm_dim)  # Critical projection

    def forward(self, image, text_ids):
        # Extract visual features
        vision_features = self.vision_encoder(image)  # [B, 576, 1024]

        # Project to LLM space
        vision_tokens = self.mm_proj(vision_features)  # [B, 576, 4096]

        # Insert at specific position
        vision_token_ids = torch.full((text_ids.shape[0], 576),
```

```
                    self.vision_token_id, device=text_ids.device)

    input_ids = torch.cat([text_ids[:, :pos], vision_token_ids,
                    text_ids[:, pos:]], dim=1)

    return self.llm(input_ids)
```

Section 3: Unified Multimodal Models

ImageBind (Girdhar 2023) - 6 Modalities:

text

Image → Audio → Text → Depth → Thermal → IMU

Emergent zero-shot: Audio → image retrieval

Chameleon (Team 2024) - Tokenize Everything:

text

Any modality → discrete tokens → unified LLM

Unified Tokenization:

text

Image: VQ-VAE → 256 tokens

Audio: EnCodec → 4 tokens/sec

Video: TimesFormer + VQ

Any-to-Any Implementation:

python

```
class UnifiedMultimodal(nn.Module):
  def __init__(self):
    self.tokenizers = {
      'image': ImageTokenizer(vocab_size=8192),
      'audio': AudioTokenizer(),
      'video': VideoTokenizer()
    }
    self.unified_llm = LlamaForCausalLM(vocab_size=32000)
```

```python
    def encode_any(self, data, modality):
        tokens = self.tokenizers[modality].encode(data)
        modality_emb = self.modality_embedding[modality]
        return tokens + modality_emb

    def generate_crossmodal(self, input_data, input_mod, output_mod):
        input_tokens = self.encode_any(input_data, input_mod)
        output = self.unified_llm.generate(input_tokens)
        return self.tokenizers[output_mod].decode(output)
```

Section 4: Audio and Video Integration

Audio Processing Pipeline:

text

1. Audio → Log-Mel spectrogram
2. AST (Audio Spectrogram Transformer)
3. Or Whisper → text → LLM

Video Understanding:

text

1. Frame sampling (1/8 FPS)
2. Space-Time patches
3. VideoMAE pretraining

Speech-to-Speech Translation:

python

```python
class SpeechAgent:
    def __init__(self, stt_model, llm, tts_model):
        self.stt = stt_model  # Whisper
        self.llm = llm
        self.tts = tts_model  # Vall-E, SpeechT5

    def translate(self, audio_input, source_lang, target_lang):
        # STT
```

```
    text = self.stt.transcribe(audio_input, source_lang)

    # Translation + adaptation
    prompt = f"Translate to natural {target_lang}: {text}"
    translated = self.llm.generate(prompt)

    # TTS
    audio_out = self.tts.synthesize(translated, target_lang)
    return audio_out
```

Section 5: Multimodal Fine-Tuning and PEFT

Alignment Techniques:

text

```
1. Captioning: Image → text (BLIP-style)
2. VQA: Image + QA pairs
3. Instruction: "Describe this chart", "Explain diagram"
```

Multimodal LoRA:

python

```
lora_config = LoraConfig(
   r=16,
   target_modules=["q_proj", "v_proj", "mm_proj"],  # Vision projector too
   modules_to_save=["mm_proj"]  # Train projection layer
)
```

LLaVA-1.5 Training Recipe:

text

```
Stage 1: Vicuna text-only warmup
Stage 2: 558k filtered LAION-GPT4V pairs
Stage 3: 150k high-res GPT4V pairs
```

Section 6: Production Deployment Patterns

Multimodal Serving:

text

1. Async preprocessing (image/audio resize)

2. Model sharding (vision + LLM)

3. Dynamic batching by modality

Cost Optimization:

text

CLIP: $0.001/query

LLaVA: $0.05/1k tokens + vision

Target: <$0.10/complex multimodal query

Real-time Video Agent:

python

```python
class VideoAgent:
    def __init__(self, vision_llm):
        self.model = vision_llm
        self.frame_buffer = deque(maxlen=32)  # 4 sec @ 8fps

    async def process_stream(self, video_stream):
        while True:
            frame = await video_stream.get_frame()
            self.frame_buffer.append(frame)

            if len(self.frame_buffer) == 32:
                analysis = await self.analyze_video()
                yield analysis
```

Section 7: Evaluation and Case Studies

Multimodal Benchmarks:

text

VQA-v2, OK-VQA, ScienceQA, ChartQA

MMBench, MME, MathVista

Case Studies:

1. **GPT-4V**: OCR + diagram reasoning
2. **Gemini 1.5**: 1M token context video
3. **Kosmos-2**: Grounding (point to objects)

Capstone: Multimodal Customer Support

Screenshot → "What's wrong?" → Diagnosis + fix instructions